Gaussian Splatting — progression notes

A from-scratch PyTorch implementation of Gaussian Splatting for 2D images and 3D scenes, plus an extension reconstructing the Eiffel Tower from internet photos. Code and instructions: `github.com/maxtlee/GaussianSplatting`.

What this is

Classical image representations store pixels. Gaussian Splatting stores a cloud of translucent 3D blobs instead: each has a position, a shape, a color, and an opacity, and a renderer projects them onto the image plane and blends them front to back. Because every step is differentiable, a rough cloud can be optimized directly against reference images until it reproduces them. This project builds that pipeline from scratch: first in 2D (fit one image), then in 3D (reconstruct a scene from posed photos), and finally on a real, messy photograph set.

The representation

A 2D Gaussian is a center μ, a covariance Σ, a color c, and an opacity α. Optimizing the four numbers of a 2×2 covariance directly is dangerous — most matrices are not valid covariances — so Σ is built from a scale and a rotation, $\Sigma = R\,S\,S^T\,R^T$, which is positive semi-definite by construction. The weight a Gaussian puts on a pixel is its value under the Gaussian density, $w(x) = \exp(-\tfrac{1}{2}(x-\mu)^T\Sigma^{-1}(x-\mu))$, and its alpha there is opacity · w. In 3D the only changes are one more position and scale axis, a rotation stored as a unit quaternion, and a projection step (below).

Rendering

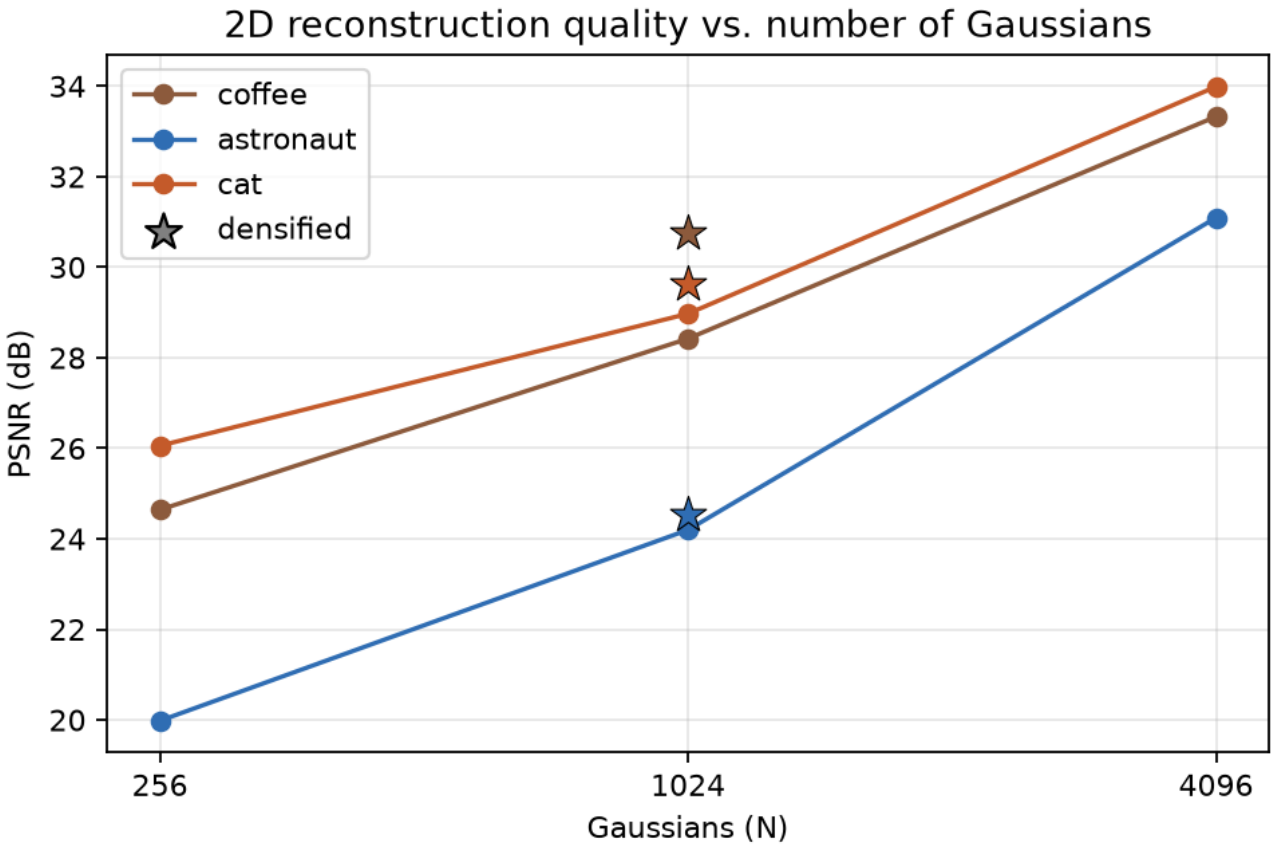
To render, the Gaussians are sorted front to back and composited with the "over" operator, $C = \sum_i c_i\alpha_i T_i$ with $T_i = \prod_{j<i}(1-\alpha_j)$, over a black background. The trick that makes this fast enough in PyTorch is to never loop over Gaussians: the transmittance T is the exclusive prefix product of $(1-\alpha)$ along the sorted axis (a single `cumprod`) and the color sum is one matrix multiply. The whole composite is a handful of vectorized operations, and gradients flow back to every position, scale, rotation, color, and opacity.

Fitting an image

Each of three 256×256 targets is fit independently with a fixed number of Gaussians, Adam at learning rate 1e-2 for 2000 steps, starting from a spread-out random initialization. PSNR is read directly off the exact full-image mean squared error.

target	N = 256	N = 1024	N = 4096	densified (N = 1024)
coffee	24.64	28.42	33.33	30.74

astronaut	19.98	24.20	31.09	24.54
cat	26.06	28.97	33.99	29.62



Final PSNR against the number of Gaussians. Stars are densified fits at their final count.

Every target improves with more Gaussians, but the ordering is not simply "busier image, more Gaussians". The astronaut needs the most capacity: it is the worst at 256 (19.98 dB) and gains the most by 4096 (+11.1 dB), because it is full of sharp, high-contrast structure — helmet edge, white suit against a dark background — and smooth blobs fit sharp edges worst. The cat is almost all high-frequency fur yet scores highest at low counts (26.06 dB), because dense texture has a small per-pixel amplitude around a dominant local color, which blobs approximate well even when the fine detail is blurred. Structure (edges and contrast) matters as much as frequency content.

Growing the cloud

A fixed cloud spends the same capacity everywhere. Densification grows it adaptively: every 200 steps each Gaussian is scored by the mean magnitude of the loss gradient with respect to its position since the last pass, and the top fraction is densified — a small under-fit Gaussian is cloned in place, a large one is split into two children offset along its major axis with scales divided by 1.6. Nearly transparent Gaussians are pruned and the count is capped

at a budget. The optimizer's Adam moments are resized alongside the parameters (survivors keep theirs, new Gaussians start at zero).

On coffee, growing 256 → 1024 reaches 30.74 dB versus 28.42 dB for a plain fixed fit at the same 1024. Densification beats the same-count fixed fit on all three targets: coffee +2.32 dB, cat +0.65 dB, astronaut +0.34 dB.

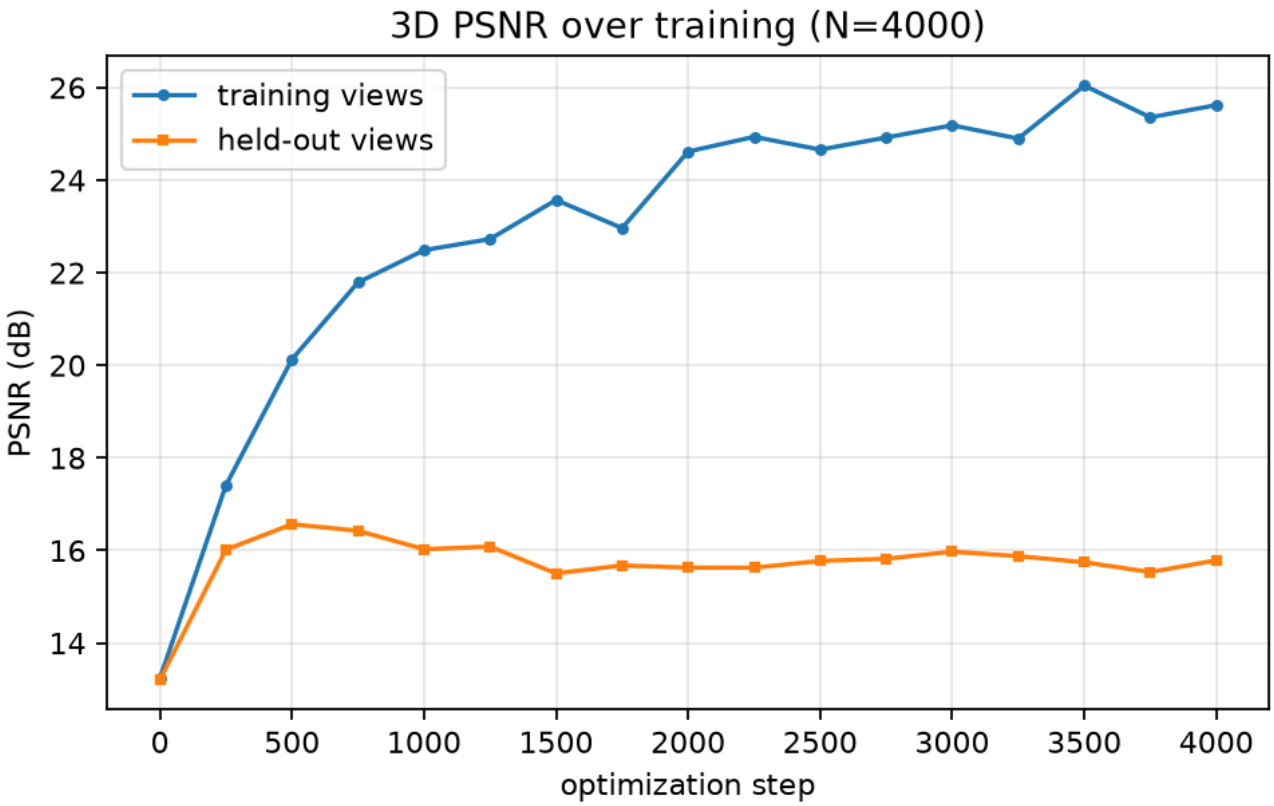


Densified coffee fit: target, render, amplified error (30.74 dB at 1024 Gaussians).

Going to 3D

Each 3D Gaussian is projected into a camera by sending its mean through a pinhole projection and its covariance through the projection's Jacobian J at the mean, $\Sigma' = J\,\Sigma_{cam}\,J^{T}$. That is the standard way to move a Gaussian through the nonlinear perspective divide, and it turns a 3D blob into a stretched 2D ellipse the 2D rasterizer can composite. Gaussians are sorted by camera-space depth z_c.

On the synthetic sphere scene (44 training views, 12 held-out), a fixed cloud of 4000 Gaussians reaches 23.60 dB on the training views and 16.51 dB on the held-out ones. A 4000-step run (evaluating every 250 steps) traces the curves below: training PSNR climbs from 13.2 dB at initialization to 25.8 dB, while held-out PSNR rises to about 16.5 dB within the first few hundred steps and then stops improving — it even drifts down slightly. The gap is overfitting: the cloud keeps fitting the training views without learning the scene.



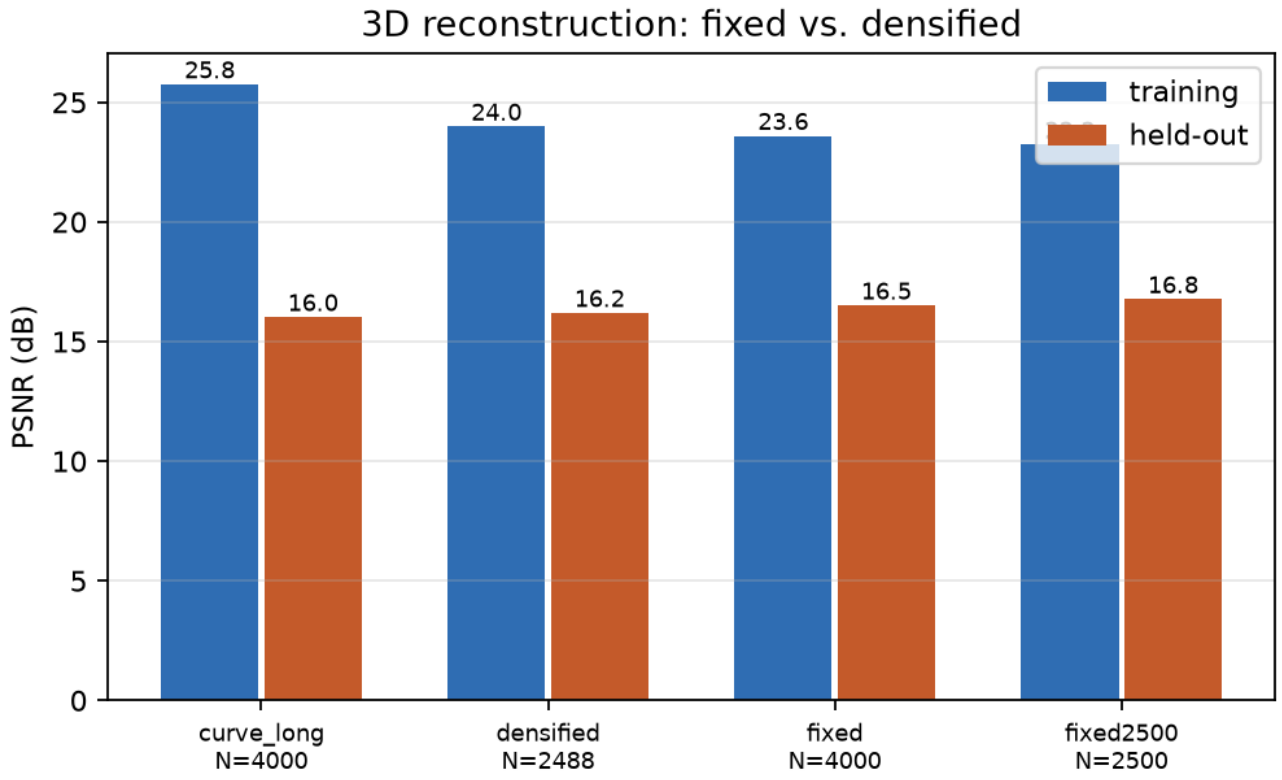
Training and held-out PSNR over 4000 optimization steps (N = 4000).



Training view 000: target, render, error. The render is recognizable but soft — the per-camera global depth sort is an approximation of the true per-pixel order, so overlapping semi-transparent primitives blend.

Densification in 3D

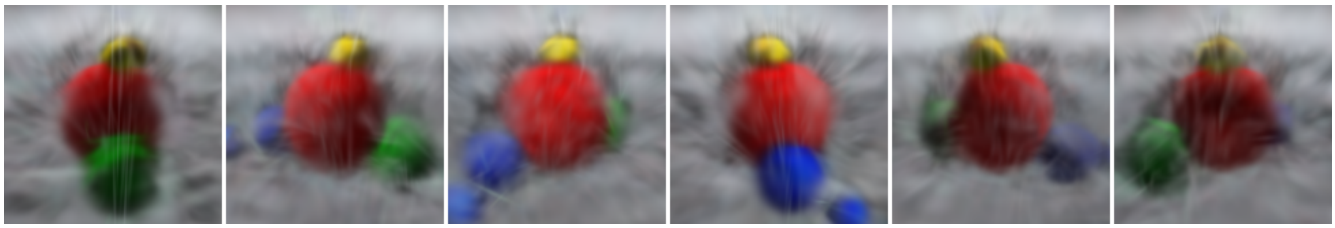
The same densification code runs in 3D with positions and scales in 3D, a quaternion rotation, a world-unit size threshold, and the position-gradient score accumulated over the sampled cameras. Here the honest result is negative: growing from 1000 to 2488 Gaussians reached a **higher training PSNR (23.99 vs 23.24 dB)** but a **lower held-out PSNR (16.20 vs 16.77 dB)** than a fixed fit at the same count. The extra primitives improve the training views, floaters included, without generalizing — exactly the failure densification is meant to prevent, and pruning did not remove enough of them at this scale.



Training and held-out PSNR for the fixed and densified sphere fits.

Novel views

Held-out PSNR (16.5 dB) is far below training PSNR (23.6 dB), so much of the fit is memorization. A 24-frame orbit renders consistently but softly: sphere silhouettes blur, contacts between spheres mush together, and the background fog drifts. This is the expected cost of a random (not structure-from-motion) initialization, a single global depth order, and view-independent color.



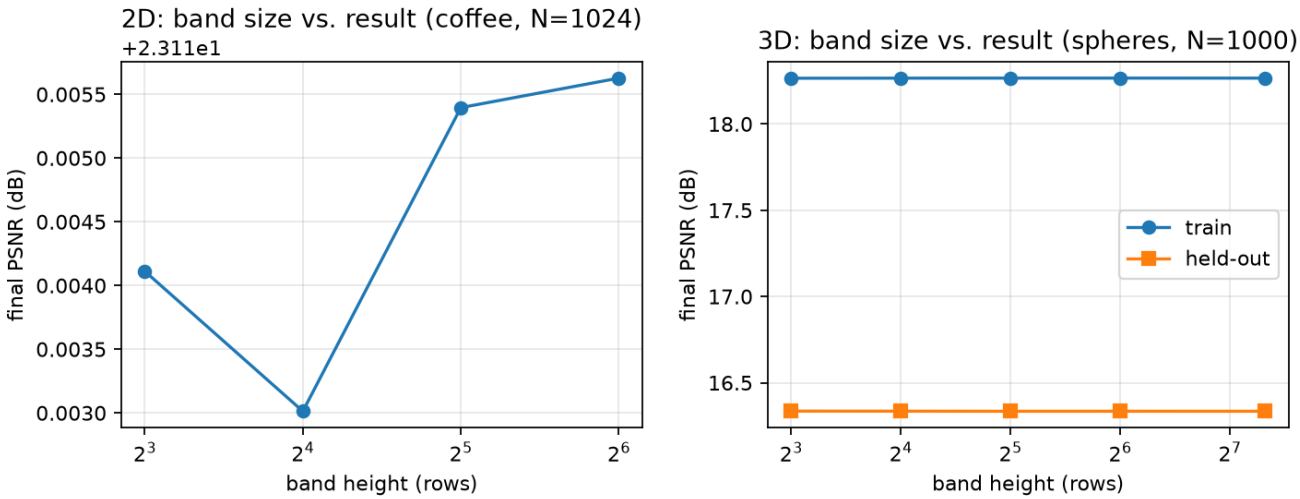
Six frames of a novel 24-view orbit (20° elevation).

Living in 4 GB: is the batching safe?

The development GPU has 4 GB of memory, which cannot hold the full (pixels × Gaussians) weight matrix (256×256 × 4096 is already > 1 GB per tensor). The renderer therefore processes the image in horizontal bands and, within a band, evaluates only the Gaussians whose support reaches it. This is meant to be a pure memory optimization, so the trained result must not depend on the band size.

It does not. Retraining the same model with band heights from 8 rows up to the whole image gives:

band (rows)	2D PSNR (coffee, N=1024)	3D train (spheres, N=1000)	3D held-out
8	23.1141	18.2650	16.3355
16	23.1130	18.2655	16.3351
32	23.1154	18.2657	16.3345
64	23.1156	18.2658	16.3347
whole image	out of memory		



Final PSNR against band height. Flat within 0.003 dB; the whole-image case does not fit.

The spread is 0.003 dB in 2D and 0.001 dB in 3D — floating-point noise. The whole-image 2D run runs out of memory, which is precisely why the bands exist. The local-support cutoff (a Gaussian is treated as zero beyond 3.5σ) is also negligible: disabling it entirely changes the held-out PSNR by 0.0002 dB. So the batching is an implementation detail, not a modeling choice.

Extension: the Eiffel Tower

To test the pipeline on real data, 320 Eiffel Tower photographs were downloaded from Wikimedia Commons (freely licensed; authors credited in `eiffel/CREDITS.md`) and reconstructed with COLMAP. The photographs have unknown poses, mixed exposures and weather, and include off-topic or unusable frames — so the pipeline adds structure-from-motion, a RANSAC selection of the consistent camera ring, and lighting compensation both before and during training. In progress — results and the orbit GIF will be added here once the reconstruction finishes.

Conclusions

- The representation and rasterizer are simple and fully differentiable; quality scales smoothly with the number of Gaussians.
- Densification helps a lot in 2D (+0.3 to +2.3 dB at equal count) but did not improve held-out quality in this 3D scene — a useful negative result.
- The 3D renders are soft; the main causes are the global depth order and the random initialization, both simplifications relative to a full tiled splatting rasterizer.
- Memory banding is provably harmless: results are identical across band sizes to within floating-point noise, and the unbanded variant does not fit.

Log

Day 1 — parsed the assignment, gathered the data, set up the CUDA 12.6 environment (the dev GPU is an old Pascal card that the default CUDA 13 wheel no longer supports).
Day 2 — implemented the 2D Gaussian, the compositing rasterizer with a `cumprod` prefix product, a resizable Adam, and the densification pass; fit the three targets.
Day 3 — added 3D covariance from quaternions and the pinhole projection with its Jacobian; reconstructed the sphere scene; found and fixed two empty-band edge cases; ran the batching study and a 5000-step 3D run.
Next — the Eiffel Tower extension.